Analysis of Unstructured Data

Lab assignments

Janusz Szwabiński

Conventions. Each problem is tagged **[Theory]** (pen-and-paper / derivation), **[Python]** (coding, submit a runnable notebook) or **[Analysis]** (interpretation, submit a short written discussion). Unless stated otherwise, submit one `.ipynb` per sheet with text answers in Markdown cells and all figures reproducible from the notebook. State every assumption; a correct derivation with a wrong final constant earns more than an unexplained correct answer.

Datasets. Prefer small, public datasets (scikit-learn / `torchvision` / `librosa` samples, 20 Newsgroups, Fashion-MNIST, CIFAR-10, Speech Commands, ESC-50). Keep everything CPU-runnable where possible; a GPU is optional.

Problem Set 1. Vector representations, similarity & dimensionality reduction

Goal: establish the "everything is a vector/tensor" paradigm and the geometry it lives in.

1. **[Theory]** Let $x, y \in \mathbb{R}^d$. Show that if $\|x\|_2 = \|y\|_2 = 1$ then $\|x - y\|_2^2 = 2\,(1 - \cos\theta)$, where $\cos\theta$ is the cosine similarity. Deduce that on the unit sphere Euclidean and cosine geometry induce the same ranking of neighbours.

2. **[Theory]** Define the *cosine distance* $d_{\cos}(x, y) = 1 - \cos\theta(x, y)$. Give an explicit counterexample in $\mathbb{R}^2$ showing that $d_{\cos}$ violates the triangle inequality, hence is not a metric. Contrast with the angular distance $\arccos(\cos\theta)/\pi$.

3. **[Theory]** (*Curse of dimensionality.*) For n points drawn i.i.d. uniformly in $[0, 1]^d$, argue that the relative gap between the nearest and farthest point from a fixed query, $\big(D_{\max} - D_{\min}\big)/D_{\min}$, tends to 0 as $d \to \infty$. Separately, show the fraction of the volume of a d-ball within an ε-shell of its surface, $1 - (1 - \varepsilon)^d$, tends to 1. Comment on the consequences for nearest-neighbour search.

4. **[Theory]** (*PCA three ways.*) Let $X \in \mathbb{R}^{n \times p}$ be column-centred with SVD $X = U\Sigma V^\top$. Prove that the direction w (with $\|w\| = 1$) maximising the sample variance $\mathrm{Var}(Xw)$ is the top right singular vector v_1, and that this coincides with the leading eigenvector of the covariance $\frac{1}{n}X^\top X$. Show further that the rank-k PCA projection minimises the reconstruction error $\sum_i \|x_i - \hat{x}_i\|_2^2$.

5. **[Python]** Load one short text document, one image and one audio clip and turn each into a NumPy array. Report the shape and dtype of each, and write two sentences per modality explaining what a "feature vector" would mean in each case.

6. **[Python]** Implement PCA *from scratch* via the SVD and reproduce `sklearn.decomposition.PCA` on the `digits` dataset (agree to 10^{-6} on the explained-variance ratios). Plot the cumulative explained variance and the 2-D projection coloured by class.

7. **[Python]** Run t-SNE and UMAP on the same features. Produce the three 2-D embeddings side by side (PCA, t-SNE, UMAP).

8. **[Analysis]** Compare the three embeddings from Problem 7: which one preserves global distances, which preserves local neighbourhoods, and why can t-SNE/UMAP distances *not* be

read as similarities? Relate your answer to Problems 1–3.

Problem Set 2. Neural networks & optimisation

Goal: derive the deep-learning backbone once, so it can be reused for text, images and audio.

1. **[Theory]** (*Softmax + cross-entropy.*) For logits $z \in \mathbb{R}^K$, softmax $p_k = e^{z_k} / \sum_j e^{z_j}$ and one-hot target y, show that the cross-entropy loss $L = -\sum_k y_k \log p_k$ has gradient $\partial L / \partial z = p - y$. Explain why this clean form is the reason softmax and cross-entropy are always paired.

2. **[Theory]** (*Backpropagation.*) Consider a two-layer MLP $h = \sigma(W_1 x + b_1)$, $\hat{y} = \mathrm{softmax}(W_2 h + b_2)$ with cross-entropy loss. Derive $\partial L / \partial W_2$, $\partial L / \partial b_2$, $\partial L / \partial W_1$, $\partial L / \partial b_1$ explicitly using the chain rule, for a general elementwise activation σ.

3. **[Theory]** Write the update rules of SGD, SGD+momentum and Adam. Show that momentum is an exponential moving average of gradients, and explain the purpose of Adam's bias-correction terms $\hat{m}_t, \hat{v}_t$ in the first few steps.

4. **[Python]** Implement the two-layer MLP of Problem 2 in *pure NumPy*, including the backward pass. Verify every gradient by central finite differences (gradient check to $< 10^{-5}$ relative error).

5. **[Python]** Re-implement the same network in PyTorch with `autograd`. Train it on the Problem-Set-1 feature set and reproduce the NumPy loss curve to confirm the implementations agree.

6. **[Python]** Study regularisation: train with/without L2, dropout and early stopping; produce train/validation learning curves for each and a small table of final accuracies.

7. **[Analysis]** Using the curves from Problem 6, identify where over- and under-fitting occur and argue which regulariser helped most on this dataset and why.

Problem Set 3. Text I/II: classical representations, classification & embeddings

Goal: move from sparse count vectors to dense distributional semantics.

1. **[Theory]** Derive the TF-IDF weight $w_{t,d} = \mathrm{tf}_{t,d} \cdot \log \frac{N}{\mathrm{df}_t}$. Give an information-theoretic reading of the $\log(N/\mathrm{df}_t)$ term and explain why raw term frequency alone is a poor weighting.

2. **[Theory]** (*Multinomial Naïve Bayes.*) Derive the MAP parameter estimates with Laplace smoothing, $\hat{\theta}_{t|c} = \dfrac{n_{t,c} + \alpha}{\sum_{t'} n_{t',c} + \alpha |V|}$. Show that the resulting decision rule is *linear* in the word-count vector, and compare its form to logistic regression.

3. **[Theory]** (*Skip-gram with negative sampling.*) Write the SGNS objective $\log \sigma(v_c^\top v_w) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n} \log \sigma(-v_{w_i}^\top v_w)$. Derive the gradient w.r.t. a centre-word vector v_w, and explain informally why SGNS approximates a shifted PMI factorisation.

4. **[Python]** Build a TF-IDF pipeline (spaCy/NLTK preprocessing → `TfidfVectorizer`) and train both multinomial Naïve Bayes and logistic regression on a text-classification dataset (e.g. 20 Newsgroups subset). Report precision/recall/F1 and list the most informative features per class.

5. **[Python]** Train word2vec with `gensim` on the same corpus and separately load pretrained GloVe. For both, return the nearest neighbours of five query words and evaluate on an analogy set (e.g. `king−man+woman`).

6. **[Analysis]** Probe the embeddings for social bias (e.g. gendered analogies or association tests). Report what you find in one paragraph and comment on the risk of propagating such bias downstream.

Problem Set 4. Text III: attention & transformers

Goal: understand and reproduce the attention mechanism, then use a pretrained transformer.

1. **[Theory]** Derive scaled dot-product attention $\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\big(QK^\top/\sqrt{d_k}\big)V$. Give the variance argument for the $1/\sqrt{d_k}$ scaling (what happens to the softmax as d_k grows without it?).

2. **[Theory]** Show that self-attention is permutation-equivariant, i.e. permuting the input rows permutes the output rows identically. Explain why this forces the use of positional encodings, and analyse the sinusoidal encoding's ability to represent relative positions.

3. **[Theory]** Count the parameters and the asymptotic compute of one multi-head attention layer with h heads and model dimension d, and compare with a dense layer of the same width. Justify the $O(n^2 d)$ cost in sequence length n.

4. **[Python]** Implement scaled dot-product attention and multi-head attention from scratch and check your output against `torch.nn.MultiheadAttention` on random inputs (max abs. error $< 10^{-5}$).

5. **[Python]** Use a pretrained HuggingFace transformer two ways: (a) as a frozen sentence-embedding extractor feeding a linear classifier, and (b) fine-tuned end-to-end. Compare both against the TF-IDF baseline from Problem Set 3 on the same data.

6. **[Analysis]** Visualise the attention weights of one head for two example sentences and discuss whether the pattern is interpretable. Note at least one caveat about reading meaning into attention maps.

Problem Set 5. Images I/II: convolution, classical features & CNNs

Goal: connect hand-crafted image features to learned convolutional features.

1. **[Theory]** Give the discrete definitions of convolution and cross-correlation for 2-D signals and prove (i) linearity and (ii) shift-equivariance. State precisely how the two operations differ and why deep-learning "convolution" is really cross-correlation.

2. **[Theory]** Derive the output-size formula for a conv layer with input size W, kernel k, padding p, stride s and dilation δ. Then count the trainable parameters of a conv layer with C_{in} input and C_{out} output channels, and compare with the equivalent dense layer.

3. **[Theory]** Describe the Sobel operator and show how HOG builds a descriptor from histograms of gradient orientations. Which geometric/photometric invariances does HOG have, and which does it lack?

4. **[Python]** Implement 2-D convolution from scratch (valid and "same" padding) and verify it against `scipy.signal.convolve2d`. Apply a blur and the Sobel edge detector to a sample image.

5. **[Python]** Compute HOG features (scikit-image) for a small image dataset and train an SVM / logistic classifier on them; record the accuracy as the *classical baseline*.

6. **[Python]** Train a CNN in PyTorch on Fashion-MNIST or CIFAR-10 with data augmentation. Report the confusion matrix and compare accuracy and training cost against the HOG baseline from Problem 5.

7. **[Analysis]** Perform an error analysis on the CNN: show the most confidently wrong examples and the classes most often confused. Quantify the effect of augmentation with an ablation.

Problem Set 6. Images III: transfer learning & modern architectures

Goal: reuse pretrained representations and understand what makes deep vision nets trainable.

1. **[Theory]** (*Residual connections.*) For a residual block $y = x + F(x)$, compute $\partial y/\partial x$ and explain, via the chain rule across many blocks, how the identity path mitigates vanishing gradients compared with a plain stack $y = F(x)$.

2. **[Theory]** (*Batch normalisation.*) Write the train-time normalisation and the learnable γ, β, and state precisely how the eval-time statistics differ. Give one reason BN can accelerate optimisation.

3. **[Theory]** Show that a Vision Transformer's patch-embedding step is a strided convolution, and connect the rest of the ViT to the attention derivation of Problem Set 4. Contrast the inductive biases of CNNs and ViTs and predict which needs more data.

4. **[Python]** Fine-tune a pretrained `torchvision` ResNet on a small custom dataset. Compare two regimes: (a) frozen backbone + trained head (feature extraction) and (b) full fine-tuning. Plot a data-efficiency curve (accuracy vs. number of training images).

5. **[Python]** Produce Grad-CAM (or a comparable saliency) visualisation for several test images and check whether the network attends to sensible regions.

6. **[Analysis]** Discuss when transfer learning helps and when it hurts, using your results and the idea of domain shift. Give one concrete situation where fine-tuning the whole backbone would be a mistake.

Problem Set 7. Audio: signals, spectral features & machine learning

Goal: turn a waveform into features and then classify it.

1. **[Theory]** State the sampling theorem and define the Nyquist frequency. Derive the DFT $X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n/N}$ and sketch the divide-and-conquer argument giving the FFT's $O(N \log N)$ cost. Explain aliasing in one sentence.

2. **[Theory]** Define the STFT and the spectrogram and state the time–frequency resolution trade-off controlled by the window length. Then write the MFCC pipeline (mel filterbank $\to$ log $\to$

DCT) and explain what role the DCT plays (decorrelation / energy compaction).

3. **[Python]** With `librosa`, load an audio clip and produce: the waveform, the magnitude spectrum (FFT), the spectrogram, the mel-spectrogram and MFCCs, each as a labelled figure. Demonstrate aliasing by deliberately down-sampling below Nyquist and describing the artefact.

4. **[Python]** Train two classifiers on a small audio dataset (e.g. Speech Commands or an ESC-50 subset): (a) MFCC features + a classical model, and (b) a log-mel spectrogram + a small CNN (the "spectrogram-as-image" approach). Compare accuracy and discuss.

5. **[Analysis]** Explain in one page the alignment problem in sequence labelling and the intuition behind CTC, and contrast keyword spotting with full ASR in terms of what each model must output.

Problem Set 8. Cross-cutting: self-supervised, multimodal & scalable retrieval

Goal: representation learning without labels, shared embedding spaces, and search at scale.

1. **[Theory]** (*Linear autoencoder* $\equiv$ *PCA.*) For a linear autoencoder with tied weights and squared reconstruction loss, show that the optimal encoder spans the same subspace as the top principal components. State where the equivalence breaks once nonlinearities are added.

2. **[Theory]** (*Contrastive learning.*) Write the InfoNCE loss for a positive pair among N candidates, derive its gradient w.r.t. the query embedding, and explain the role of the temperature τ. State in what sense InfoNCE lower-bounds mutual information.

3. **[Theory]** (*Scalable search.*) For signed feature hashing, show that the hashed inner product is an unbiased estimator of the true inner product and give its variance's dependence on the number of buckets. For cosine LSH with random hyperplanes, show the collision probability is $1 - \theta/\pi$.

4. **[Python]** Train a small autoencoder on an image or feature dataset; visualise the latent space and use the reconstruction error as an anomaly score on held-out corrupted samples.

5. **[Python]** Build a semantic / cross-modal search demo: embed a corpus with a pretrained sentence- or CLIP-style model, index it with FAISS, and measure recall@k and query latency against exact brute-force search as the index size grows.

6. **[Analysis]** Write a one-page reflection tying the course together: pick one application area, name the main evaluation pitfalls (class imbalance, annotation quality, leakage) and one ethical concern (bias, privacy, or misuse) specific to unstructured data, and state how you would mitigate it.